# System do zarządzania więzieniami

Karol Niekrasz

S27969

Modelowanie i architektura systemów informacyjnych

## Spis treści

System do zarządzania więzieniami ..... 1

1.Wymagania Użytkownika ..... 1

2.Diagram przypadków użycia ..... 3

3.Diagram klas ..... 4

4. Diagram klas – projektowy ..... 5

5. Scenariusz przypadku użycia: Zarządzanie personelem więzienia ..... 6

6.Diagram aktywności dla przypadku użycia ..... 7

7.Diagram stanu klas ..... 8

8.Projekt GUI ..... 8

9.Omówienie decyzji projektowych ..... 11

## 1.Wymagania Użytkownika

System do zarządzania więzieniami ma na celu usprawnienie i zautomatyzowanie zarządzania tą instytucją. Użytkownikami systemu będą pracownicy administracji więziennej, strażnicy, oraz personel zarządzający. Oczekiwania wobec systemu obejmują:

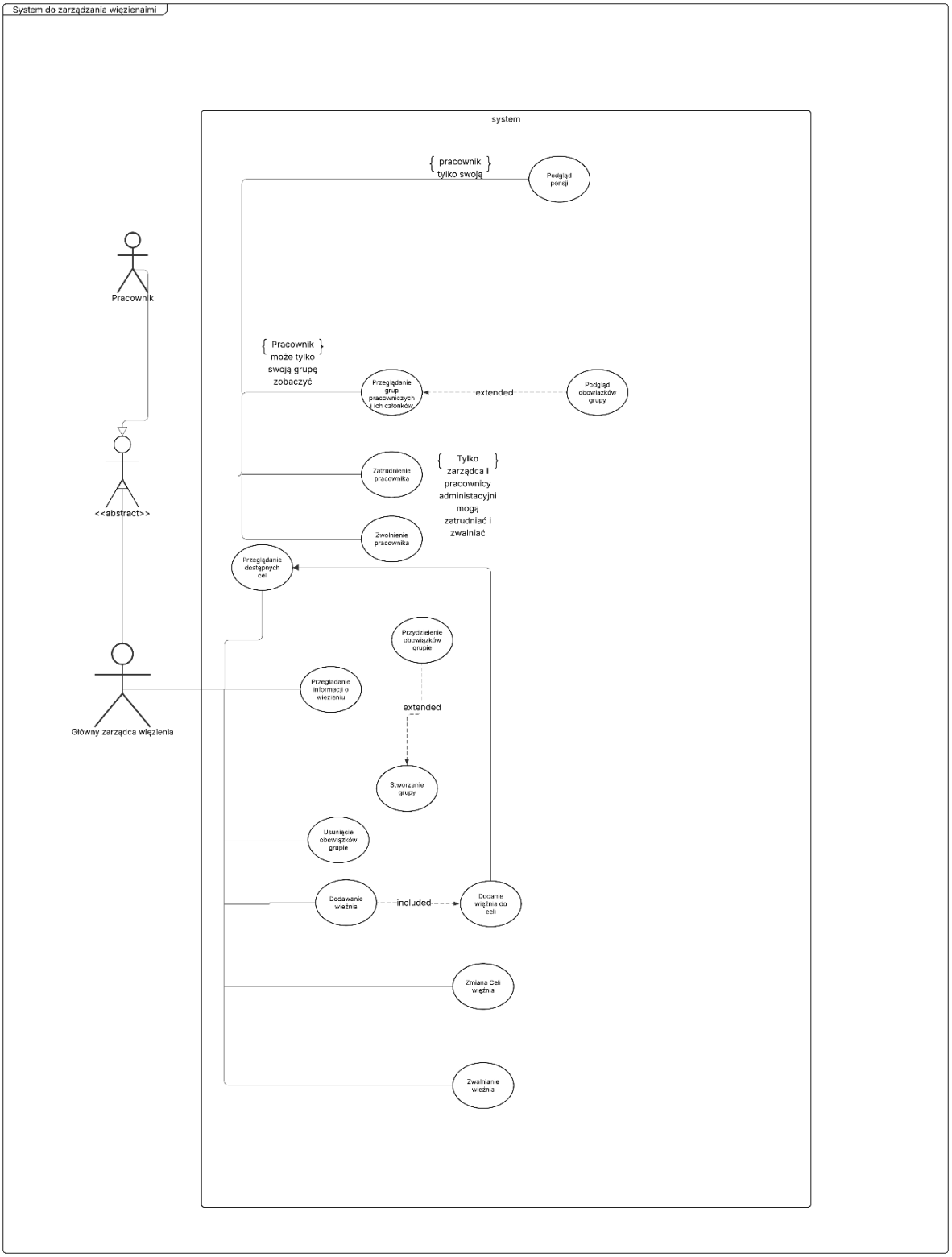
- **Rejestracja i zarządzanie więźniami**:
    - Dodawanie nowych więźniów do systemu wraz z ich danymi osobowymi, historią wyroków i przydziałem do celi.
    - Przeglądanie i edytowanie informacji o osadzonych.
    - Sprawdzenie wyroków oskarżonych
    - Dodawanie wyroków oskarżonym
    - Zwolnienie warunkowe więźniów

- **Zarządzanie celami**:
    - Przegląd dostępnych cel oraz liczby osadzonych w każdej z nich.
    - Automatyczne przydzielanie cel na podstawie dostępności.
- **Zarządzanie personelem**:
    - Zatrudniać i zwalniać pracowników
    - Powiązanie pracowników z przypisanymi obowiązkami i grupami
    - Dodawanie i zmienianie obowiązków poszczególnych grup
    - Awansować pracowników, poprzez dodanie im licencji na broń lub uczynienie szefem danej grupy
- **Zarządzanie flotą pojazdów więzienia**
    - Dodawanie oraz usuwanie pojazdów z parkingu więziennego
- **Raportowanie i analiza**:
    - Sprawdzenie danych poszczególnych więzień i informacji na ich temat
    - Sprawdzanie stopnia niebezpieczeństwa danego więzienia

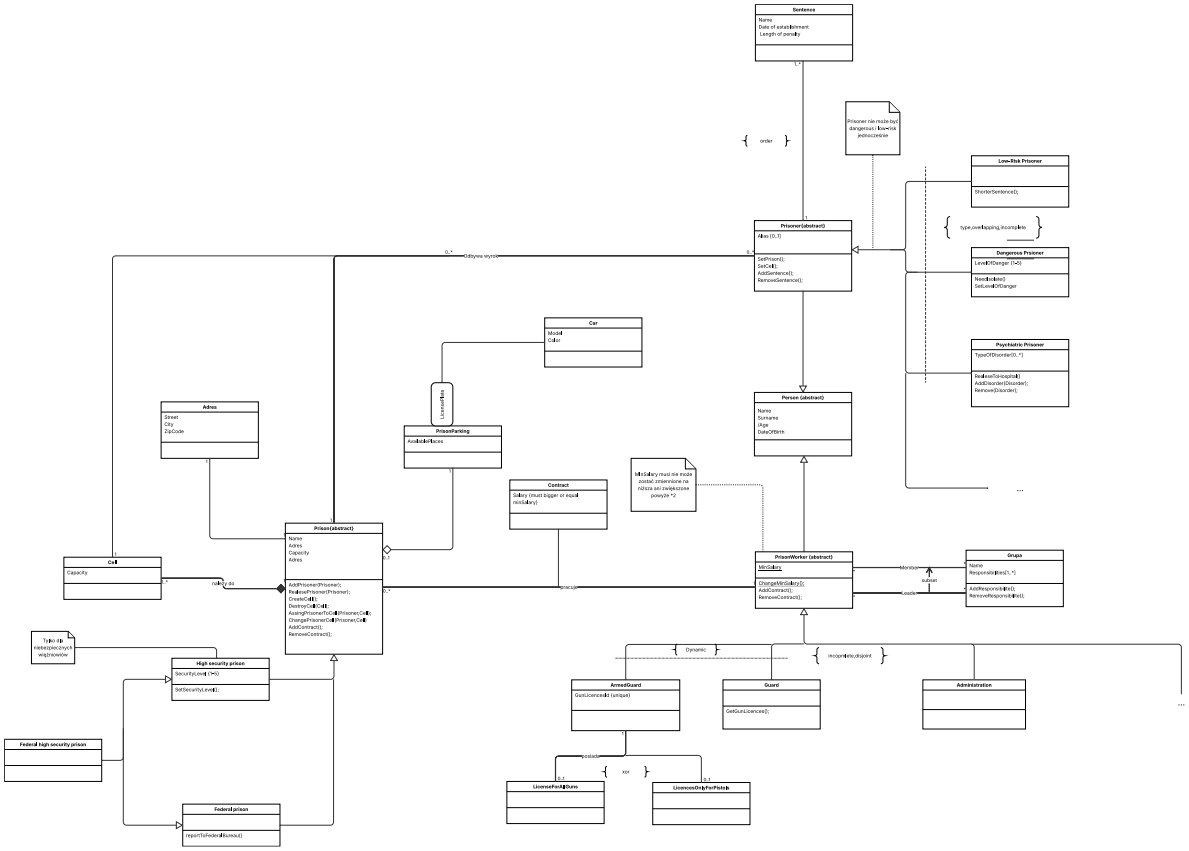
## Wymagania Techniczne

System powinien również zapewnić trwałość danych, przy użyciu bazy relacyjnej lub innego typu wspieranego przez Entity Framework. System również powinien być łatwo rozszerzalny i przygotowany do przyszłej ewentualnej rozbudowy.

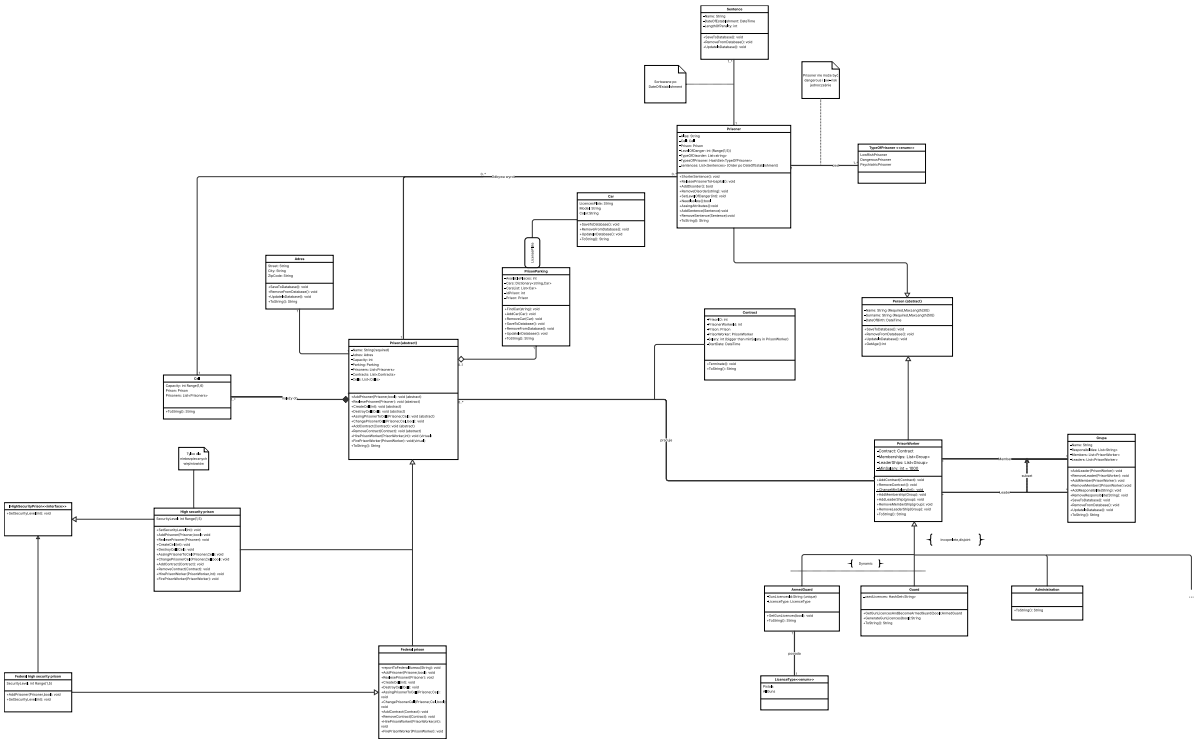
# 2.Diagram przypadków użycia



# 3.Diagram klas



# 4. Diagram klas – projektowy



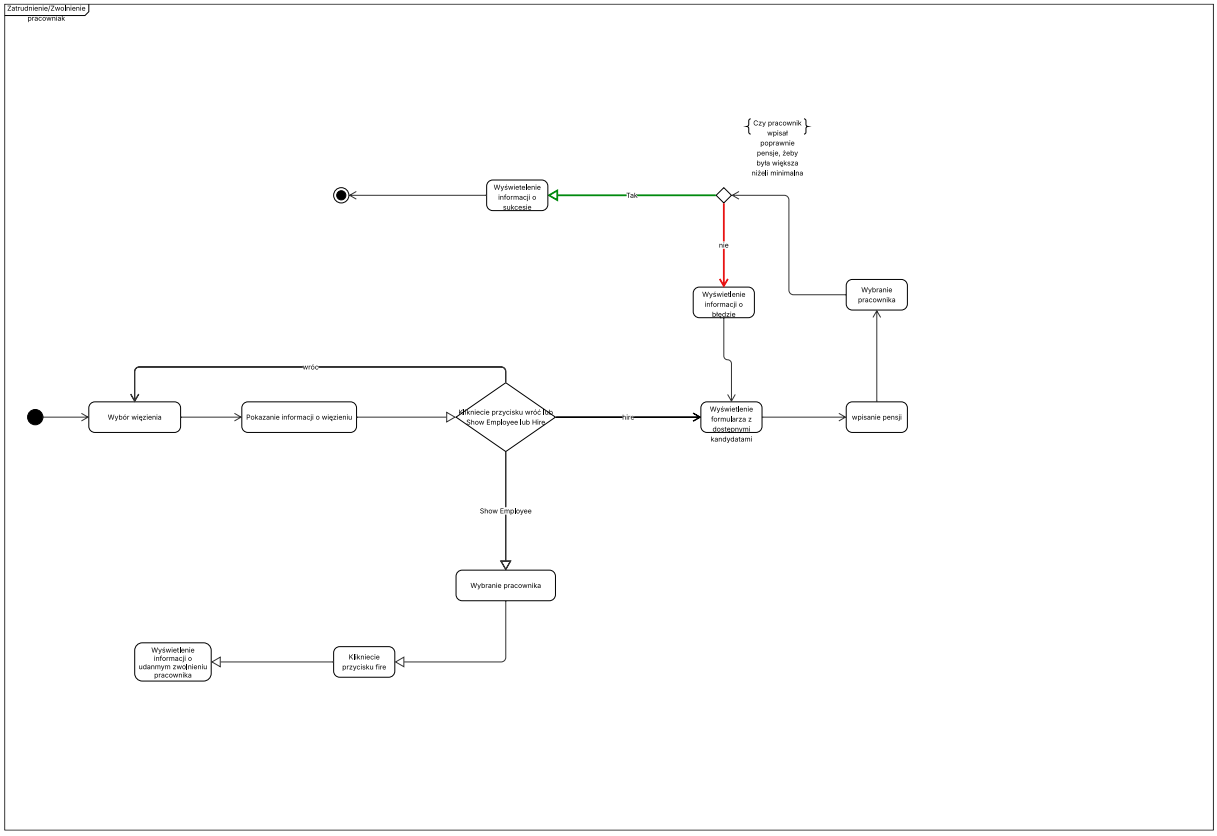
## 5. Scenariusz przypadku użycia (jako tekst)

Zarządca więzień uruchomił system i skierował swoją uwagę na listę dostępnych więzień. Wybrał interesujące go z listy. Wyświetliły mu się informacje o danym więzieniu.

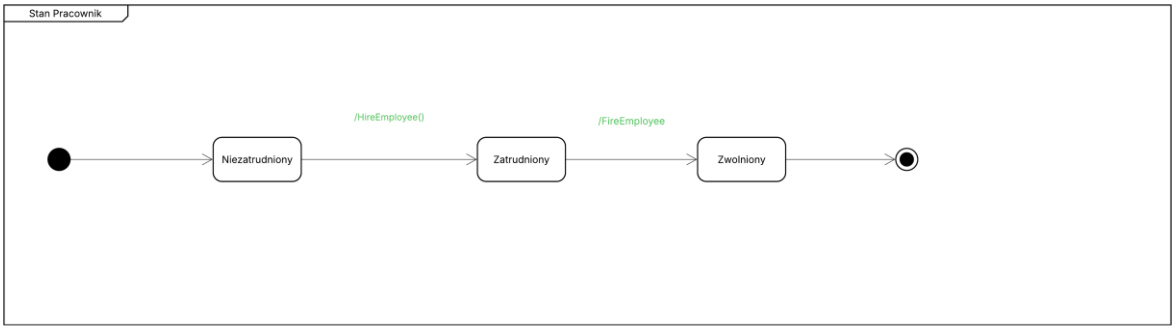
Następnie mógł kliknąć przycisk „Show Employee” lub Hire, jeśli kliknął ten pierwszy, mógł podejrzeć pracowników więzienia i ich dane. Tam zarządca mógł wybrać ewentualnego pracownika z listy i kliknąć przycisk „Fire”, jeśli chciał go zwolnić. System wyświetlił wtedy komunikat o zwolnieniu pracownika. Tym samym pracownik został usunięty z więzienia i trafił do puli pracowników, których wszystkie inne więzienia mogą zatrudnić.

Natomiast miał jeszcze tam drugi wybór i przycisk „Hire”, po kliknięciu, którego wyświetliła mu się lista wolnych pracowników, zarządca musiał wybrać pracownika z listy oraz wpisać jego pensje. Potem zarządca mógł kliknąć przycisk „Hire” i zatrudnić pracownika. Natomiast w tym momencie system waliduje pensje pracownika i jeśli zarządca podał pensje niższą niż minimalna system wyświetla informacje o potrzebie zmiany jej. Natomiast jeśli zarządca ustawił dobrą pensje system wyświetli informacje o udanym zatrudnieniu.

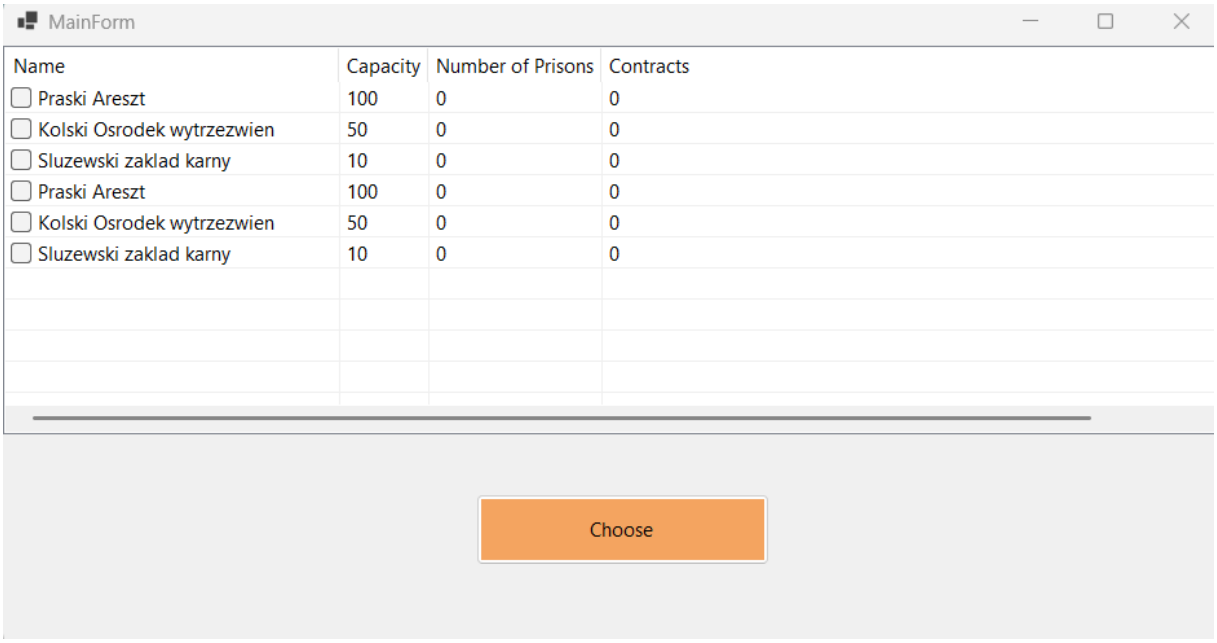
# 6.Diagram aktywności dla przypadku użycia

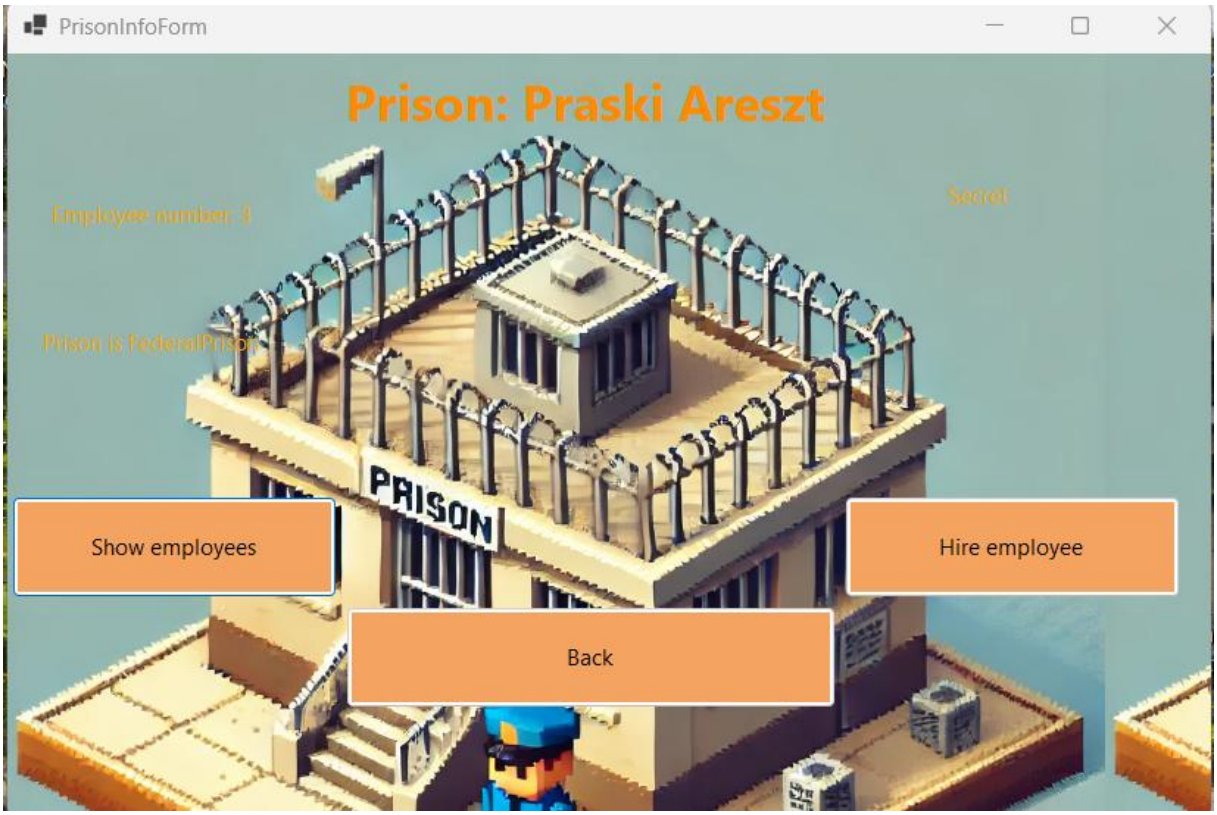


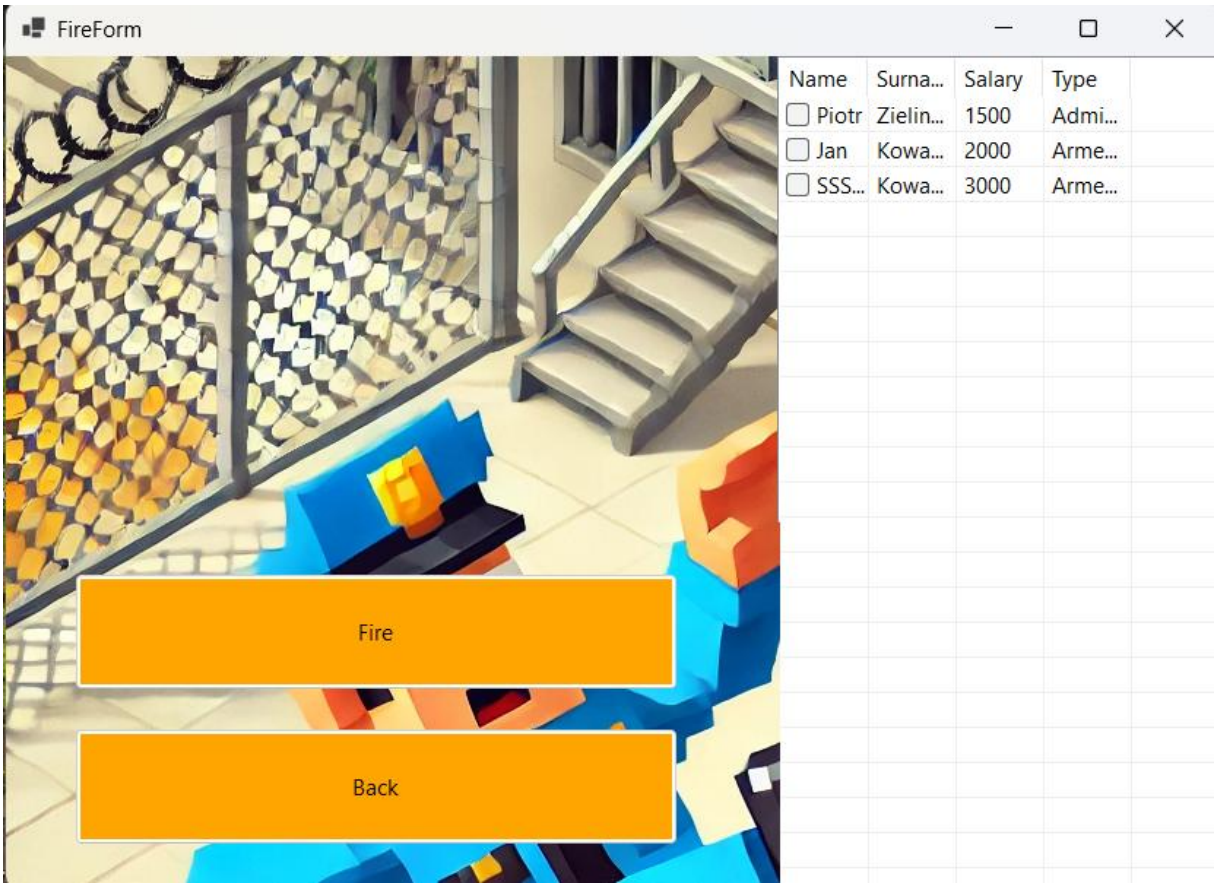
## 7.Diagram stanu klas




## 8.Projekt GUI




PrisonInfoForm
Prison: Praski Areszt
Employee number: 1
Secret
Prison is FederalPrison
PRISON
Show employees
Hire employee
Back


FireForm
Name
Surna...
Salary
Type
Piotr
Zielin...
1500
Admi...
Jan
Kowa...
2000
Arme...
SSS...
Kowa...
3000
Arme...
Fire
Back

HireForm
Name
Surna...
Age
Type
Jan
Kowa...
40
Arme...
Piotr
Zielin...
36
Admi...
Jan
Kowa...
40
Arme...
Input salary:
Hire
Back

## 9.Omówienie decyzji projektowych

Decyzje projektowe zostały podjęte po dokładnej analizie wymagań funkcjonalnych oraz struktury danych systemu.

**Asocjacje**

Asocjacje pomiędzy obiektami zostały zaimplementowane przy użyciu natywnych referencji obiektów. Pozwala to na szybki i bezpośredni dostęp do danych, w przeciwieństwie do podejścia opartego na identyfikatorach, które wymuszałoby ręczne wyszukiwanie odpowiednich instancji obiektów w kolekcjach.

**Asocjacja z atrybutem**

Relację pomiędzy obiektem Prison a PrisonWorker, która zawiera dodatkowe informacje (np. pensja), zrealizowałem poprzez klasę pośredniczącą Contract. Takie podejście umożliwia modelowanie relacji zawierającej dane dodatkowe.

**Kwalifikowana asocjacja**

Kwalifikowaną asocjację pomiędzy PrisonParking a Car zaimplementowałem z wykorzystaniem mapy (np. Dictionary w C#), gdzie kluczem jest identyfikator samochodu. Takie podejście pozwala na szybkie wyszukiwanie pojazdów przypisanych do parkingu danego więzienia.

**Kompozycja**

Kompozycję zastosowałem pomiędzy klasami Prison i Cell. Każde więzienie zarządza swoją listą cel. W momencie usunięcia instancji klasy Prison, automatycznie usuwane są również wszystkie przypisane do niej obiekty Cell. Klasy te nie są zagnieżdżone, lecz stanowią osobne encje.

**Wielodziedziczenie**

Z uwagi na brak bezpośredniego wsparcia dla wielodziedziczenia w C#, zastosowałem połączenie interfejsu IHighSecurityPrison z klasą bazową FederalPrison. W rezultacie powstała klasa FederalHighSecurityPrison, która łączy cechy obu struktur.

**Dziedziczenie rozłączne (overlapping)**

W przypadku klas więźnia (np. PsychiatricPrisoner, DangerousPrisoner) zamiast tworzenia osobnych klas, zastosowałem uproszczone rozwiązanie: spłaszczenie hierarchii i użycie typu wyliczeniowego (enum). Rozwiązanie to jest prostsze w implementacji i bardziej wydajne.

**Ograniczenie unikalności (UNIQUE)**

W celu zapewnienia unikalności identyfikatorów GunLicenceID, użyłem mapy przechowującej aktualnie nadane identyfikatory. Podczas tworzenia nowej licencji system sprawdza dostępność identyfikatora i przydziela nowy w przypadku kolizji.

**Kolejność elementów (ORDER)**

Wyroki (Sentence) przypisane do więźnia są uporządkowane chronologicznie według dat. Choć nie było możliwości zapewnienia tego porządku bezpośrednio na poziomie SQL, to w warstwie aplikacyjnej (DbContext w C#) kolekcja wyroków jest zawsze sortowana według daty przed dalszym przetwarzaniem.

**Ograniczenie dynamiczne – Guard / ArmedGuard**

Ograniczenie dynamiczne pomiędzy klasami Guard oraz ArmedGuard zaimplementowałem poprzez stworzenie dwóch osobnych klas, z których obie dziedziczą po klasie bazowej PrisonWorker.
W momencie „przekształcenia" zwykłego strażnika (Guard) w uzbrojonego strażnika (ArmedGuard), tworzony jest nowy obiekt klasy ArmedGuard, a istniejący obiekt Guard jest usuwany.

**XOR**

Zaimplementowałem w klasie Guard, za pomocą prostego warunku, który zakazuje posiadania dwóch enum-ów z nazwami broni przez klase Guard.

## Skutki analizy dynamicznej

Podczas analizy skupiłem się na interakcji systemu z użytkownikiem i jego działaniem.

Zauważyłem, że system potrzebuję walidacji danych wpisywanych przez użytkownika i sensownych komunikatów, przez to zaimplementowałem specjalne kontrolki informujące użytkownika o złych decyzjach.

Kolejnym ważnym aspektem jest synchronizacja danych. Tutaj musiałem zaimplementować zarządzanie połączeniem z naszą bazą danych. Niestety w aplikacjach Winforms nie działa to do końca dobrze, gdyż kontekst był stworzony do szybkich operacji na danych i ich zamykania. Co bardziej sugeruje nam architekturę MVC lub po prostu operacje przeniesione do Repozytoriów metod związanych z tworzeniem obiektów i zarządzaniem nimi. Niestety żeby rozwiązać ten problem musiałem zaimplementować globalny statyczny kontekst, który musi być zamykany w momencie zamknięcia ostatniego okienka przez EventHandler.

Dostrzegłem też możliwość uproszczenia troszkę implementacji Id do bazy danych poprzez dodanie klasy BasicEntity, tutaj pomyślałem również o rozszerzeniu jej o klasę AuditableEntity, która zawiera daty modyfikacji w bazie i osoby za to odpowiedzialne.